

Capítulo 3 Clasificación

A diferencia de la regresión, donde se busca predecir un número, la clasificación intenta determinar a qué categoría pertenece una determinada información. Para explicar estos conceptos se utiliza el dataset **MNIST**, formado por 70.000 imágenes de dígitos escritos a mano. Cada imagen tiene 28×28 píxeles, por lo que puede representarse mediante 784 características con valores de intensidad entre 0 y 255. Se comienza con un problema sencillo: identificar si una imagen corresponde al número 5 o no (**clasificación binaria**). Se usa **SGDClassifier**, un algoritmo basado en Descenso de Gradiente Estocástico que procesa las observaciones progresivamente y puede trabajar eficientemente con grandes cantidades de datos.

Accuracy o exactitud no siempre es una buena métrica; MNIST, aproximadamente solo una décima parte de las imágenes corresponde al número 5. Por lo tanto, un modelo que siempre respondiera “no es 5” podría obtener una exactitud superior al 90% sin realmente ser de utilidad, demostrando el problema de las clases desbalanceadas y la importancia de utilizar métricas más adecuadas.

Para analizar los resultados se utiliza **matriz de confusión**, que permite observar los aciertos y errores del clasificador y se obtienen verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos. Con esos valores se calculan principalmente la **precisión**, que indica qué proporción de los positivos predichos realmente son positivos, y el **recall o exhaustividad**, que muestra qué proporción de los positivos reales fue detectada. Estas métricas normalmente presentan un intercambio: aumentar el recall puede provocar una disminución de la precisión y viceversa. Este equilibrio se conoce como *trade-off* entre precisión y recall.

El **F1-score** combina precisión y recall mediante una media armónica, por lo que resulta útil cuando se desea equilibrar ambas métricas. No siempre es la mejor opción, porque dependiendo del problema puede ser más importante minimizar los falsos positivos o los falsos negativos.

Otra herramienta importante es **curva ROC** que relaciona la tasa de verdaderos positivos con la tasa de falsos positivos para diferentes umbrales de decisión. Su desempeño puede resumirse mediante el **AUC**, donde un valor cercano a 1 representa un excelente clasificador y 0,5 corresponde aproximadamente a un clasificador aleatorio.

También se explica que cuando existe un fuerte desbalance entre clases, la curva precisión/recall puede proporcionar una evaluación más útil que la ROC.

Posteriormente se pasa a la **clasificación multiclase** donde existen más de 2 categorías. Se presentan dos estrategias principales:

- *One-versus-All* (OvA), que entrena un clasificador para cada clase,

- *One-versus-One* (OvO), que entrena clasificadores para cada par de clases. La estrategia más conveniente depende del algoritmo utilizado y de su comportamiento con grandes cantidades de datos.

Se destaca **análisis de errores**. La matriz de confusión permite descubrir qué clases se confunden entre sí y buscar posibles soluciones; en MNIST, por ejemplo, algunos números pueden ser difíciles de distinguir debido a pequeñas diferencias en su escritura. Poder analizar esos errores puede llevar a conseguir más datos, crear nuevas características o mejorar el procesamiento de imágenes.

Los conceptos de **clasificación multilabel y multioutput**:

- multilabel, una misma observación puede recibir varias etiquetas simultáneamente.
- multioutput, cada salida puede tener diferentes valores posibles, como ocurre al intentar eliminar ruido de una imagen prediciendo nuevamente la intensidad de cada píxel.

Conclusión, el capítulo demuestra que construir un buen clasificador no consiste solamente en obtener un porcentaje alto de exactitud. Es fundamental elegir las métricas de acuerdo con el problema, entender el costo de los falsos positivos y falsos negativos, analizar la matriz de confusión y estudiar errores específicos de el modelo. Las herramientas permiten evaluar de manera más realista el desempeño de un sistema de clasificación y encontrar formas concretas de mejorar.

Capítulo 4 entrenamiento de modelos

Se explica con mayor profundidad cómo funcionan matemáticamente algunos de los modelos fundamentales de ML. Después de haber utilizado algoritmos como cajas negras, el objetivo ahora es comprender qué ocurre internamente cuando un modelo aprende. Lo que sirve como base para entender posteriormente el funcionamiento de las redes neuronales.

En **Regresión Lineal** se presentan dos formas principales de encontrar los parámetros que minimizan el error. La primera es la **Ecuación Normal**, que permite calcular directamente la solución óptima minimizando el error cuadrático medio. Aunque es exacta, puede resultar muy costosa cuando aumenta considerablemente el número de características. La segunda alternativa es el **Descenso de Gradiente**, un método iterativo que comienza con valores iniciales y va modificando los parámetros en la dirección que reduce el error, la rapidez del proceso depende de la **tasa de aprendizaje**:

- demasiado pequeña, el entrenamiento puede ser muy lento,
- demasiado grande puede impedir que el algoritmo converja correctamente.

El Descenso de Gradiente tiene tres variantes principales.

- **Batch Gradient Descent** utiliza todos los datos disponibles para calcular cada actualización, es estable pero puede ser lento.
- **Stochastic Gradient Descent (SGD)** utiliza una sola observación en cada paso, rápido y adecuado para grandes cantidades de datos, pero resultados presentan más variación.
- **Mini-batch Gradient Descent** utiliza pequeños grupos de observaciones y representa un punto intermedio entre los 2 anteriores.

Estas técnicas son especialmente importantes porque también se utilizan para entrenar redes neuronales.

Se introduce la **Regresión Polinomial**, una técnica que permite representar relaciones no lineales agregando potencias de las variables originales, como x^2 o x^3 , y entrenando posteriormente un modelo lineal sobre esas nuevas características. Sin embargo, aumentar demasiado el grado del polinomio puede producir un número excesivo de características y favorecer sobreajuste.

Para detectar **underfitting y overfitting** se presentan las curvas de aprendizaje, que comparan el error de entrenamiento con el error de validación mientras aumenta la cantidad de datos. Un modelo con errores altos y similares en ambos conjuntos suele presentar underfitting, mientras que una diferencia grande entre ambos errores puede indicar overfitting. Esto se relaciona con el **trade-off sesgo-varianza**: un modelo demasiado simple suele tener alto sesgo y poca capacidad para representar los datos, mientras que uno demasiado complejo puede tener alta varianza y ser excesivamente

sensible a los datos de entrenamiento. Además, existe un error irreducible provocado por el ruido presente en los propios datos.

Para reducir el sobreajuste se presentan diferentes técnicas de **regularización**:

- **Ridge Regression** utiliza una penalización basada en los cuadrados de los pesos y reduce su magnitud.
- **Lasso Regression** utiliza los valores absolutos y puede llevar algunos pesos exactamente a cero, realizando así una especie de selección automática de características.
- **Elastic Net** combina las ventajas de Ridge y Lasso.
- **Early Stopping**, consiste en detener el entrenamiento cuando el desempeño sobre los datos de validación comienza a empeorar, evitando que el modelo continúe aprendiendo ruido.

Regresión Logística se utiliza principalmente para clasificación binaria y el modelo transforma una combinación de las características mediante la función sigmoide, obteniendo un valor entre 0 y 1 que puede interpretarse como una probabilidad. Después, según un umbral de decisión, se determina la clase correspondiente (su entrenamiento utiliza una función de costo llamada *log loss* o entropía cruzada).

Regresión Softmax o regresión logística multinomial, extiende este concepto a problemas con varias clases. El modelo calcula una puntuación para cada clase y utiliza la función Softmax para convertirlas en probabilidades que suman uno. Finalmente, selecciona la clase con mayor probabilidad.

Conclusión, se presentan fundamentos matemáticos esenciales para comprender cómo aprenden los modelos. El Descenso de Gradiente, la regularización, el sesgo-varianza y las funciones utilizadas en la regresión logística serán conceptos fundamentales en capítulos siguientes.

La principal idea es que comprender cómo se entrenan y cómo se controlan los modelos permite diagnosticar problemas como underfitting y overfitting y elegir estrategias adecuadas para mejorar su rendimiento.

